

# Smart Complaint Management System: Software Requirements, Test Plan, and Architecture & Design Specification

Software Engineering Mini-Project — Deliverables Part-1

Narina Jeevan Naga Deep

M Nikhil Sai

Nandakishore PV

N Kanaka Mani Sai Akhil

PES1UG24CS293

PES1UG24CS253

PES1UG24CS291

PES1UG24CS290

Department of Computer Science and Engineering, PES University, Bengaluru, India

**Abstract**—Complaints handled manually are slow to organise and opaque to the people who raise them. The Smart Complaint Management System (SCMS) is a web platform on which users register, submit and track complaints and give feedback, while administrators view, categorise, assign, update and resolve them. This report consolidates the Part-1 deliverables: a Software Requirements Specification following IEEE 830 / ISO/IEC/IEEE 29148, a Test Plan following IEEE 829 with sixteen test cases, and a Software Architecture and Design Specification following IEEE 1016 / ISO/IEC/IEEE 42010, all linked through a requirements traceability matrix.

**Index Terms**—complaint management, SRS, test plan, layered architecture, RBAC, REST API, traceability.

## I. INTRODUCTION

### A. Problem Statement

Managing complaints manually is difficult and time-consuming. Users often do not know the status of their complaints, and administrators struggle to organise, assign and track them. SCMS provides one centralised platform where users submit and track complaints and administrators manage, assign, update and resolve them efficiently.

### B. Purpose, Scope and Audience

This document specifies *what* SCMS must do (Section II), *how it will be verified* (Section III) and *how it is structured* (Section IV). SCMS 1.0 covers account management, complaint lifecycle management, feedback and administrative handling. Notifications by e-mail/SMS and native mobile apps are out of scope. The audience is the development team, course evaluators and testers.

### C. Definitions and Acronyms

**FR/NFR**: functional / non-functional requirement; **SR**: security requirement; **RBAC**: role-based access control; **JWT**: JSON Web Token; **DAL**: data access layer; **RTM**: requirements traceability matrix; **UAT**: user acceptance testing.

### D. Feasibility Study

**Technical**: buildable with standard web technologies and MySQL. **Economic**: relies on free, open-source tools, so cost is low. **Operational**: the interface is simple for both users and administrators. **Schedule**: a four-member team can build and test it within the project timeline. The system is therefore technically, economically, operationally and schedule feasible.

TABLE I  
ACTORS AND USE CASES

| ID | Actor         | Use cases                                                                                                 |
|----|---------------|-----------------------------------------------------------------------------------------------------------|
| A1 | User          | Register, Login, Submit Complaint, Track Complaint, Provide Feedback                                      |
| A2 | Administrator | Admin Login, View Complaints, Categorize Complaint, Assign Complaint, Update Complaint, Resolve Complaint |

## II. SOFTWARE REQUIREMENTS SPECIFICATION

### A. Overall Description

1) *Product perspective*: SCMS is a new, self-contained web application: a browser client talks over HTTPS to a REST API server, which persists data in a MySQL database.

2) *User classes*: **A1 User** registers, submits and tracks complaints and gives feedback. **A2 Administrator** views, categorises, assigns, updates and resolves complaints (Table I).

3) *Operating environment*: Latest two versions of Chrome, Firefox and Edge on desktop and mobile; server on Ubuntu 22.04 with Node.js 20 and MySQL 8.

4) *Constraints and assumptions*: Only open-source tools are used; departments are pre-configured by the administrator; users have a valid e-mail address and Internet access.

### B. Use Case Model

Fig. 1 shows the two actors and eleven use cases inside the system boundary. Each use case maps to at least one FR in Table II and is traced to tests in Table VIII.

### C. Functional Requirements

The informal list of fourteen requirements was split into eight FRs and six NFRs, and each was rewritten as a single, verifiable “shall” statement (Tables II and III).

### D. Non-Functional Requirements

Table III states each quality attribute with a measurable target so that it can be verified by a specific test.

### E. External Interface Requirements

*User interface*: responsive web pages with separate User and Administrator dashboards. *Software interfaces*: REST/JSON API (Section IV-G) and MySQL 8 via a pooled driver. *Communication*: HTTPS (TLS 1.2 or higher) only.

### F. Security Requirements

#### Security objectives.

- **SO1 Confidentiality**: only authorised parties can read user and complaint data.
- **SO2 Integrity**: complaint records and status history cannot be altered by unauthorised actors.
- **SO3 Accountability**: every administrative action is at-

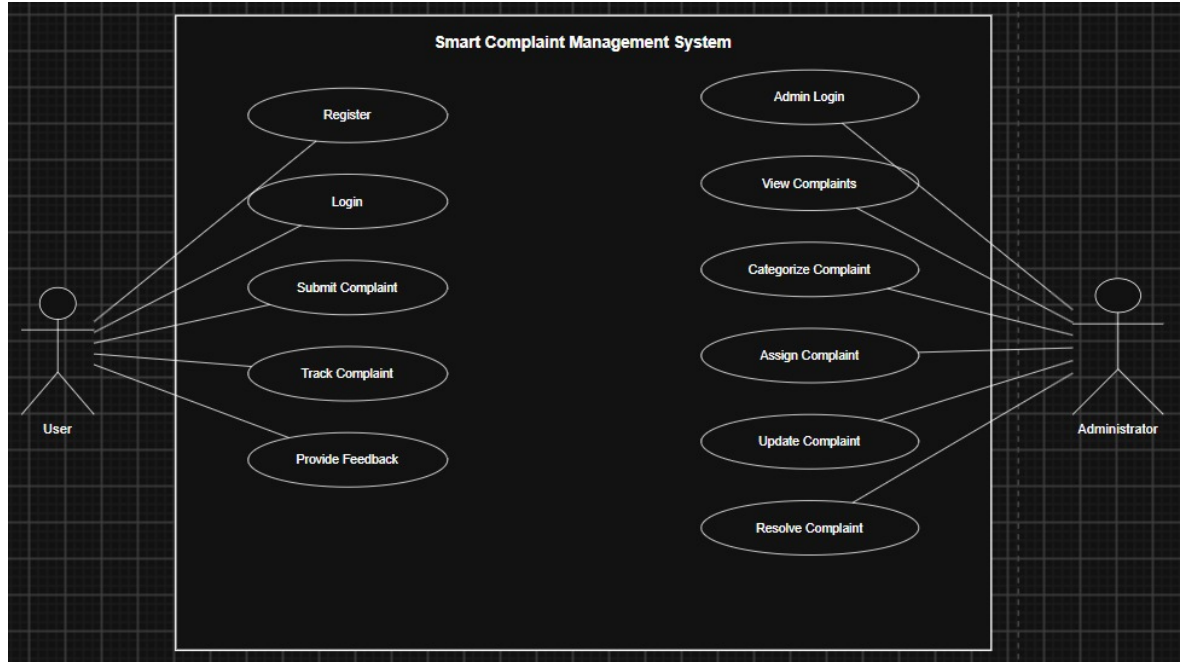


Fig. 1. UML use case diagram of the Smart Complaint Management System showing actors User and Administrator and their use cases.

TABLE II  
FUNCTIONAL REQUIREMENTS

| ID   | Requirement (the system shall ...)                                                                                                                                                                                        |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FR01 | let a new user register with name, unique e-mail and a password of $\geq 8$ characters containing a letter and a digit, and let users and administrators log in with e-mail and password; duplicate e-mails are rejected. |
| FR02 | let an authenticated user submit a complaint with title ( $\leq 100$ chars), description ( $\leq 1000$ chars), category and an optional JPG/PNG/PDF attachment ( $\leq 5$ MB).                                            |
| FR03 | assign every complaint a unique ID of the form CMP-YYYY-NNNNNN at submission and display it to the user.                                                                                                                  |
| FR04 | show the owner of a complaint its current status (Submitted, Assigned, In Progress, Resolved, Closed) and its timestamped status history.                                                                                 |
| FR05 | let an administrator list all complaints, filter them by status, category and date range, and set or change a complaint's category.                                                                                       |
| FR06 | let an administrator assign a complaint to exactly one department from a predefined list, changing its status to Assigned.                                                                                                |
| FR07 | let an administrator change a complaint's status only along the defined workflow, with a remark ( $\leq 500$ chars), and mark it Resolved with a resolution note; every change records time and admin ID.                 |
| FR08 | let the owner submit exactly one feedback (rating 1–5, optional comment $\leq 300$ chars) per complaint, only after it is Resolved.                                                                                       |

TABLE III  
NON-FUNCTIONAL REQUIREMENTS

| ID    | Attribute       | Measurable requirement                                                                                                                                                                               |
|-------|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NFR01 | Security        | Every non-public page and API shall require a valid session; access shall follow SR1–SR6 (Section II-F).                                                                                             |
| NFR02 | Usability       | A first-time user shall submit a complaint in $\leq 3$ min without training; every main task shall be reachable in $\leq 3$ clicks from the dashboard; layouts shall work from 360 to 1920 px width. |
| NFR03 | Performance     | 95% of login, submit and track requests shall complete in $\leq 2$ s with 100 concurrent users.                                                                                                      |
| NFR04 | Reliability     | No committed complaint data shall be lost: all writes are transactional, availability $\geq 99\%$ per month, daily backups kept for 7 days.                                                          |
| NFR05 | Privacy         | A complaint and its owner's personal data shall be visible only to that owner and to administrators; data shall travel only over HTTPS.                                                              |
| NFR06 | Maintainability | Code shall be layered so one module changes without editing others; service-layer unit-test coverage $\geq 70\%$ ; zero ESLint errors.                                                               |

tributable to a specific administrator.

#### Security requirements.

- **SR1:** Passwords shall be stored only as bcrypt hashes (cost  $\geq 10$ ) and never logged.
- **SR2:** Sessions shall use signed JWTs that expire 60 minutes after issue.

- **SR3:** RBAC and ownership checks shall be enforced server-side on every API call; violations return 403/404.
- **SR4:** Five consecutive failed logins shall lock the account for 15 minutes.
- **SR5:** All inputs shall be validated server-side, all SQL shall use parameterised queries, and output shall be encoded to prevent injection and XSS.
- **SR6:** All traffic shall use HTTPS, and every admin action shall be written to an audit log (who, what, when).

SR1, SR2 and SR4 realise SO1; SR3 and SR5 realise SO1 and SO2; SR6 realises SO3.

### III. TEST PLAN

1) *Introduction:* Test plan ID *SCMS-TP-1.0*. It verifies FR01–FR08, NFR01–NFR06 and SR1–SR6 of this SRS against the design in Section IV.

2) *Test items:* SCMS 1.0 build: React web client; Node.js/Express API with Auth, Complaint, Assignment and Feedback services; middleware; DAL; MySQL schema.

3) *Features to be tested:* Registration and login (FR01); complaint submission and ID generation (FR02, FR03); tracking (FR04); admin view, filter and categorise (FR05); assignment (FR06); status workflow and resolution (FR07); feedback rules (FR08); access control, privacy and all security controls (NFR01, NFR05, SR1–SR6); response time (NFR03); data durability (NFR04); usability (NFR02); maintainability (NFR06).

4) *Features not to be tested:* E-mail/SMS notifications and native mobile apps (out of scope); loads above 100 concurrent users; browsers older than the latest two versions; infrastructure-level penetration testing of the hosting provider.

5) *Approach:* Unit tests (Jest) cover service-layer logic with  $\geq 70\%$  statement coverage. Integration tests (Supertest) call every API endpoint against a seeded test database. System tests (Selenium) run end-to-end user and admin workflows in the browser. Performance tests use Apache JMeter; usability tests use timed tasks with five new users; UAT is run by the team with sample users. Black-box design uses equiva-

lence partitioning and boundary-value analysis (e.g., 100/101-character titles, 5.0/5.1 MB files, ratings 0/1/5/6); the status workflow is covered by state-transition testing. The full regression suite runs on every merge to the main branch.

**5.1 Security validation.** *Authentication:* brute-force lock-out (SR4), expired and missing tokens (SR2), password storage inspection (SR1). *Authorisation:* vertical escalation (User token on admin APIs) and horizontal access (User A reading User B’s complaint) (SR3). *Input handling:* SQL-injection and XSS payloads in every text field (SR5). *Transport and audit:* HTTP-to-HTTPS redirect and audit-log entries for admin actions (SR6). An OWASP ZAP baseline scan is run on each release candidate; it passes when no High or Medium alerts remain.

6) *Pass/fail criteria:* A test passes when the actual result equals the expected result. Release requires 100% of security and high-priority tests to pass,  $\geq 95\%$  overall pass rate, and no open critical or high defects.

7) *Suspension and resumption:* Testing is suspended if the build fails the smoke test or the test database is unavailable, and resumes after a fixed build passes the smoke test.

8) *Environment and deliverables:* Ubuntu 22.04, Node.js 20, MySQL 8 with synthetic data (1000 complaints, 50 users); tools: Jest, Supertest, Selenium, JMeter, OWASP ZAP. Deliverables: this plan, test case specification (Table IV), test logs, defect reports and a test summary report.

9) *Responsibilities and schedule:* N.J.N.Deep: test lead and security tests; M.N.Sai: unit and integration tests; Nandakishore PV: system and usability tests; N.K.M.S.Akhil: performance and reliability tests. Week 1 unit, week 2 integration, week 3 system/security/performance, week 4 UAT and summary report.

10) *Risks:* Schedule pressure is handled by testing security and high-priority features first; real personal data is never used in testing.

**Test cases and traceability.** Table IV lists sixteen test cases: eight functional, five security, and one each for performance, reliability, usability and maintainability, so every FR, NFR and SR is covered by at least one test. The end-to-end mapping from requirements to use cases, design components and tests is given in the RTM (Table VIII).

## IV. SOFTWARE ARCHITECTURE & DESIGN SPECIFICATION

### A. Architectural Pattern

SCMS uses a **layered (three-tier) client-server architecture**: a presentation tier (React single-page app), an application tier (Node.js/Express, internally organised as routes  $\rightarrow$  middleware  $\rightarrow$  services  $\rightarrow$  DAL, an MVC-style layering) and a data tier (MySQL). Each layer only calls the layer below it. This gives separation of concerns and replaceable modules (NFR06), a single enforcement point for authentication and authorisation (NFR01, SR3), and suits a four-member team. Microservices were rejected as too much operational overhead for this scale; an unlayered monolith was rejected because it would hurt maintainability.

### B. Component Diagram, Description and Traceability

Fig. 2 shows the components; Table V describes each and traces it to the requirements it realises.

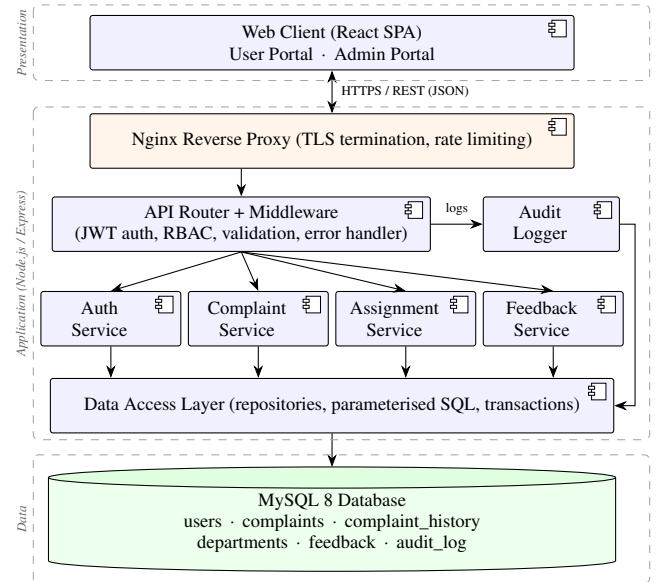


Fig. 2. UML component diagram of SCMS (three tiers).

### C. Security Architecture

Security is layered (defence in depth). *Transport:* Nginx terminates TLS 1.2+ and redirects HTTP to HTTPS (SR6). *Authentication:* the Auth Service hashes passwords with bcrypt and issues HS256-signed JWTs with a 60-minute expiry; the signing secret lives in an environment variable, never in code (SR1, SR2, SR4). *Authorisation:* middleware checks the role on every route, and the Complaint Service additionally checks ownership, returning 404 for others’ complaints so their existence is not revealed (SR3, NFR05). *Input:* schema validation in middleware, parameterised SQL in the DAL, and React output encoding (SR5). *Data:* the application uses a least-privilege database account without DDL rights, and the database is reachable only from the private network. *Accountability:* the Audit Logger stores every admin action (SO3).

### D. Other Architectural Views

*Deployment:* browser  $\rightarrow$  Nginx  $\rightarrow$  Node.js application server  $\rightarrow$  MySQL, with the last two on a private network. *Data design:* USER(user\_id, name, email, password\_hash, role, failed\_attempts, locked\_until); COMPLAINT(complaint\_id, user\_id, title, description, category, status, dept\_id, created\_at); COMPLAINT\_HISTORY(id, complaint\_id, old\_status, new\_status, remark, changed\_by, changed\_at); DEPARTMENT(dept\_id, name); FEEDBACK(complaint\_id [unique], rating, comment); AUDIT\_LOG(id, admin\_id, action, target, at). *Complaint states:* Submitted  $\rightarrow$  Assigned  $\rightarrow$  In Progress  $\rightarrow$  Resolved  $\rightarrow$  Closed (after feedback or 7 days).

### E. Design: UML Sequence Diagrams

Fig. 3 shows complaint submission (FR02, FR03) and Fig. 4 shows administrator assignment with RBAC enforcement (FR06, SR3).

### F. API Design

The API is RESTful under base path /api/v1, exchanges JSON, and expects Authorization: Bearer <JWT> on every endpoint except register and login (Table VI). List endpoints are paginated (?page=&size=, default 20). For example, POST /complaints with {title, description, category} returns 201 with {complaintId: “CMP-2026-000123”, status: “Submitted”}.

TABLE IV  
TEST CASES (FUNCTIONAL, NON-FUNCTIONAL AND SECURITY)

| TC ID | Req.            | Type               | Test description / input                                                                                                           | Expected result                                                                                                               |
|-------|-----------------|--------------------|------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| TC01  | FR01            | Functional         | Register with name "Ravi K", an unused e-mail and password "Pass1234"; then register again with the same e-mail.                   | 201 Created, account has role User; second attempt returns 409 "email already registered".                                    |
| TC02  | FR01            | Functional         | Log in with valid user credentials, then with valid administrator credentials.                                                     | JWT issued in both cases; user lands on User dashboard, admin on Admin dashboard.                                             |
| TC03  | FR02            | Functional (BVA)   | Submit complaints with 100- and 101-character titles, and with 5.0 MB and 5.1 MB attachments.                                      | 100 chars and 5.0 MB accepted; 101 chars and 5.1 MB rejected with 422 and a field-level message.                              |
| TC04  | FR02, FR03      | Functional         | Submit two valid complaints one after the other.                                                                                   | Both saved (201); IDs match CMP-YYYY-NNNNNN, are different, and are shown to the user.                                        |
| TC05  | FR04            | Functional         | Owner opens a complaint that was assigned and then moved to In Progress.                                                           | Status "In Progress" shown with three timestamped history entries.                                                            |
| TC06  | FR05, FR06, SR6 | Functional         | Admin filters status = Submitted, sets category "Electrical", assigns to "Maintenance".                                            | Only Submitted complaints listed; status becomes Assigned; history and audit-log rows store the admin ID.                     |
| TC07  | FR07            | Functional (state) | Admin moves Assigned → In Progress → Resolved with remarks, then attempts Resolved → Submitted.                                    | Valid transitions saved with remarks; invalid transition returns 409 and status is unchanged.                                 |
| TC08  | FR08            | Functional (BVA)   | Owner rates a Resolved complaint 4, then rates it again; rates an unresolved complaint; submits rating 6.                          | First feedback 201; repeat 409; unresolved 409; rating 6 returns 422.                                                         |
| TC09  | NFR01, SR2, SR3 | Security           | Call PATCH /complaints/{id}/assign with a User token; call any protected API with no token and with a token issued 61 min earlier. | 403 for the User token; 401 for missing and expired tokens; no data modified.                                                 |
| TC10  | NFR05, SR3      | Security           | User A requests GET /complaints/{id} for a complaint owned by User B.                                                              | 404 returned; response contains no field of B's complaint.                                                                    |
| TC11  | SR1, SR4        | Security           | Enter a wrong password five times, then the correct one; inspect the users table.                                                  | Account locked for 15 min (423) even with the correct password; only bcrypt hashes (\$2b\$, cost ≥ 10) stored.                |
| TC12  | SR5, SR6        | Security           | Login e-mail ' OR '1'='1; description <script>alert(1)</script>; request over plain HTTP; OWASP ZAP baseline scan.                 | Login rejected; script displayed as escaped text, never executed; HTTP answered with 301 to HTTPS; no High/Medium ZAP alerts. |
| TC13  | NFR03           | Performance        | JMeter: 100 concurrent virtual users for 10 min (60% track, 30% submit, 10% login).                                                | 95th-percentile response time ≤ 2s; error rate < 1%.                                                                          |
| TC14  | NFR04           | Reliability        | Stop the API server during a submission and restart; restore the test database from the last daily backup.                         | No partial complaint/history rows (rolled back); all committed complaints intact; restore completes successfully.             |
| TC15  | NFR02           | Usability          | Five first-time users, without training, submit and then track a complaint.                                                        | All five submit within 3 min; each task reached in ≤ 3 clicks from the dashboard.                                             |
| TC16  | NFR06           | Maintainability    | Run ESLint and Jest coverage; add a comment-length rule to the Feedback service and re-run the regression suite.                   | Zero lint errors; service coverage ≥ 70%; only Feedback-module files changed; all regression tests pass.                      |

TABLE V  
COMPONENT DESCRIPTION AND REQUIREMENT TRACEABILITY

| Component               | Responsibility                                                                         | Realises             |
|-------------------------|----------------------------------------------------------------------------------------|----------------------|
| Web Client              | User and Admin pages, client-side validation, output encoding                          | FR01–08, NFR02       |
| Nginx Proxy             | TLS termination, HTTP→HTTPS redirect, rate limiting                                    | SR6, NFR03           |
| API Router + Middleware | Routing, JWT verification, RBAC, input validation, central error handling              | NFR01, SR2, SR3, SR5 |
| Audit Logger            | Records who did what and when for admin actions                                        | SR6, FR07            |
| Auth Service            | Registration, login, bcrypt hashing, logout                                            | FR01, SR1, SR4       |
| Complaint Service       | Submission, ID generation, tracking, categorisation, status workflow, ownership checks | FR02–05, FR07, NFR05 |
| Assignment Service      | Department list and complaint assignment                                               | FR06                 |
| Feedback Service        | One feedback per resolved complaint                                                    | FR08                 |
| DAL                     | Repositories, parameterised queries, transactions                                      | NFR04, SR5           |
| MySQL DB                | Persistent storage, daily backups                                                      | NFR04                |

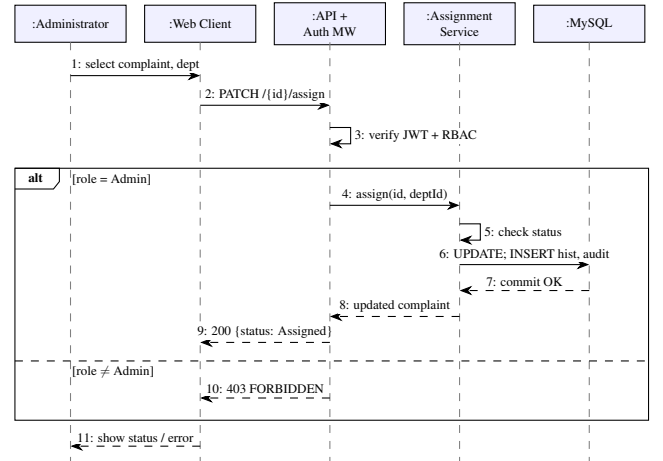


Fig. 4. Sequence diagram: Assign Complaint with RBAC check.

TABLE VI  
REST API ENDPOINTS

| Method | Endpoint                  | Role        | Purpose (success)                 |
|--------|---------------------------|-------------|-----------------------------------|
| POST   | /auth/register            | Public      | Create account (201)              |
| POST   | /auth/login               | Public      | Return JWT and role (200)         |
| POST   | /complaints               | User        | Submit, return ID (201)           |
| GET    | /complaints               | User/Admin  | Own list / all with filters (200) |
| GET    | /complaints/{id}          | Owner/Admin | Details and history (200)         |
| PATCH  | /complaints/{id}/category | Admin       | Set category (200)                |
| PATCH  | /complaints/{id}/assign   | Admin       | Assign department (200)           |
| PATCH  | /complaints/{id}/status   | Admin       | Update/resolve with remark (200)  |
| POST   | /complaints/{id}/feedback | Owner       | Rating and comment (201)          |

### G. Error Handling

All errors pass through one Express error-handling middleware and return a uniform envelope: {"error": {"code", "message", "details", "requestId"}}. Status codes are mapped as in Table VII. Database errors roll back

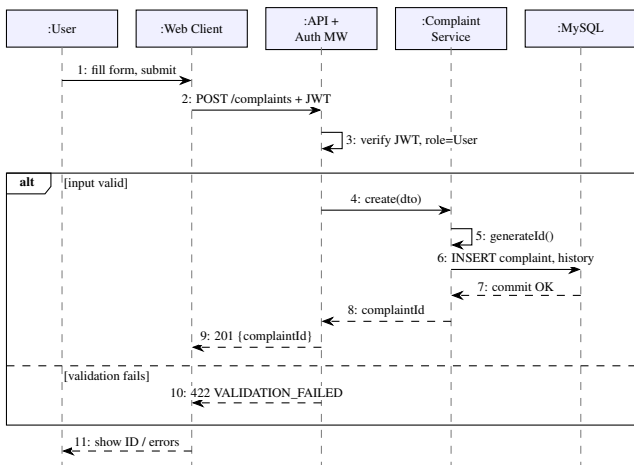


Fig. 3. Sequence diagram: Submit Complaint.

TABLE VII  
ERROR CODE MAPPING

| HTTP | Code              | When                                                |
|------|-------------------|-----------------------------------------------------|
| 400  | BAD_REQUEST       | Malformed JSON or parameters                        |
| 401  | UNAUTHENTICATED   | Missing, invalid or expired JWT                     |
| 403  | FORBIDDEN         | Role not allowed for the action                     |
| 404  | NOT_FOUND         | Resource absent or not owned by caller              |
| 409  | CONFLICT          | Duplicate e-mail/feedback, invalid state transition |
| 422  | VALIDATION_FAILED | Field rules violated (length, type, size)           |
| 423  | ACCOUNT_LOCKED    | Lockout after 5 failed logins                       |
| 429  | RATE_LIMITED      | Too many requests from one client                   |
| 500  | INTERNAL_ERROR    | Unexpected failure (details logged only)            |

TABLE VIII  
REQUIREMENTS TRACEABILITY MATRIX

| Req.     | Use case                     | Component                 | API                           | Tests    |
|----------|------------------------------|---------------------------|-------------------------------|----------|
| FR01     | Register, Login, Admin Login | Auth Service              | /auth/*                       | TC01, 02 |
| FR02     | Submit Complaint             | Complaint Svc             | POST /complaints              | TC03, 04 |
| FR03     | Submit Complaint             | Complaint Svc             | POST /complaints              | TC04     |
| FR04     | Track Complaint              | Complaint Svc             | GET /complaints/{id}          | TC05     |
| FR05     | View, Categorize             | Complaint Svc             | GET /complaints, .../category | TC06     |
| FR06     | Assign Complaint             | Assignment Svc            | .../assign                    | TC06     |
| FR07     | Update, Resolve              | Complaint Svc, Audit      | .../status                    | TC07     |
| FR08     | Provide Feedback             | Feedback Svc              | .../feedback                  | TC08     |
| NFR01    | All (access)                 | Middleware                | All protected                 | TC09     |
| NFR02    | All                          | Web Client                | —                             | TC15     |
| NFR03    | All                          | All tiers                 | All                           | TC13     |
| NFR04    | Complaint mgmt.              | DAL, MySQL                | Write APIs                    | TC14     |
| NFR05    | Track, View                  | Middleware, Complaint Svc | GET /complaints/{id}          | TC10     |
| NFR06    | Maintenance                  | Layered structure         | —                             | TC16     |
| SR1, SR4 | Register, Login              | Auth Service              | /auth/*                       | TC11     |
| SR2, SR3 | All                          | Middleware                | All protected                 | TC09, 10 |
| SR5      | Login, Submit                | Middleware, DAL           | All                           | TC12     |
| SR6      | Admin use cases              | Nginx, Audit Logger       | All                           | TC06, 12 |

the open transaction; unexpected errors are logged server-side with the request ID and returned as a generic 500 message, so stack traces and SQL never reach the client (SR5). The client shows friendly messages, highlights invalid fields, and redirects to login on 401.

#### H. Other Design Considerations

Complaint IDs come from a per-year database sequence inside the insert transaction, guaranteeing uniqueness under concurrency (FR03). Coding follows the ESLint Airbnb style guide, and work is merged through reviewed Git pull requests that must pass the regression suite.

### V. REQUIREMENTS TRACEABILITY MATRIX

Table VIII traces each requirement to its use case, design component, API and test cases, closing the loop across the SRS, the design and the test plan.

### VI. CONCLUSION

This report delivers the Part-1 artefacts for SCMS: an IEEE-format SRS with eight measurable FRs, six NFRs, a use case model and a security section with three objectives and six requirements; an IEEE 829 test plan with a dedicated security-validation approach and sixteen test cases; and an architecture

and design specification with a layered architecture, component and sequence diagrams, security architecture, API design and error-handling strategy. The RTM confirms that every requirement is realised by a component and verified by at least one test.

### REFERENCES

- [1] IEEE Recommended Practice for Software Requirements Specifications, IEEE Std 830-1998.
- [2] Systems and Software Engineering—Life Cycle Processes—Requirements Engineering, ISO/IEC/IEEE 29148:2018.
- [3] IEEE Standard for Software and System Test Documentation, IEEE Std 829-2008.
- [4] IEEE Standard for Information Technology—Software Design Descriptions, IEEE Std 1016-2009.
- [5] Systems and Software Engineering—Architecture Description, ISO/IEC/IEEE 42010:2011.
- [6] OWASP Foundation, “OWASP Top 10: 2021,” 2021.